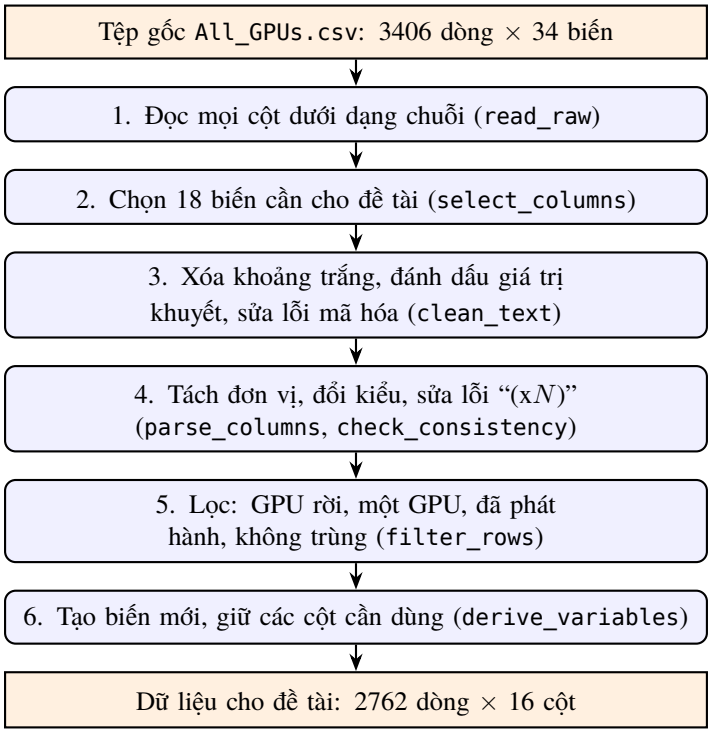
3 Tiền xử lý số liệu

Phần này trình bày quá trình chuyển tệp `All_GPUs.csv` (3406 dòng, 34 biến, mô tả ở phần tổng quan) thành bộ dữ liệu dùng cho các phân tích ở mục sau. Toàn bộ quy trình viết bằng Python (thư viện `pandas`), chạy lại được từ đầu và cho cùng một kết quả. Quy trình gồm sáu bước: đọc dữ liệu, chọn biến, làm sạch chuỗi, xử lý định dạng, lọc quan sát và tạo biến mới (Hình 1, kèm tên hàm thực hiện mỗi bước).

Nhóm tuân theo ba nguyên tắc: không sửa tay dữ liệu gốc; mỗi hàm đọc biến số chỉ chấp nhận đúng định dạng đã biết và dừng chương trình nếu gặp giá trị lạ; không điền giá trị cho biến mục tiêu `Max_Power`. Ngoại lai và biến đổi logarit được xử lý trong từng phân tích ở các mục sau, vì mỗi phân tích cần một cách xử lý khác nhau.

Tên biến trong tệp gốc viết hoa, như `Max_Power`; tên cột sau xử lý viết thường và kèm đơn vị, như `max_power_w`. Giá trị khuyết được ghi là `NA`.



Hình 1: Sơ đồ khối quy trình tiền xử lý

3.1 Bước 1: Đọc dữ liệu

Mọi cột của tệp gốc được đọc dưới dạng chuỗi, chưa chuyển đổi giá trị nào, để mọi giá trị đều đi qua bước làm sạch và kiểm tra định dạng ở bước 3 và bước 4. Mỗi dòng là một mẫu card và được gán mã `gpu_id` là vị trí của dòng trong tệp gốc, để từ một dòng trong dữ liệu cuối có thể tìm lại đúng dòng gốc.

Đoạn code 1: Đọc dữ liệu thô dưới dạng chuỗi

```
raw = pd.read_csv(path, dtype="str", keep_default_na=False)
```

Kết quả: 3406 dòng × 34 biến, toàn bộ là chuỗi, thêm cột `gpu_id`.

3.2 Bước 2: Chọn biến

Trong 34 biến gốc, nhóm chỉ giữ 18 biến cần cho câu hỏi nghiên cứu, chia thành ba loại:

- biến mục tiêu `Max_Power` và các biến giải thích `TMUs`, `Core_Speed`, `Memory`, `Memory_Bandwidth`, `Process`;
- các biến phân nhóm và định danh: `Name`, `Manufacturer`, `Notebook_GPU`, `Memory_Type`, `Release_Date`, `Architecture`;
- các biến chỉ dùng trong lúc xử lý: `Dedicated` (lọc GPU rời), `ROPs` (sửa lỗi "(xN)" ở bước 4), `Texture_Rate` (kiểm tra lỗi và tính hiệu quả năng lượng), `Boost_Clock`, `Memory_Bus`, `Memory_Speed` (kiểm tra lỗi).

Trong đó, `TMUs` (texture mapping units) là số đơn vị xử lý texture, phản ánh quy mô bộ xử lý của GPU; `ROPs` (render output units) là số đơn vị ghi điểm ảnh vào bộ nhớ khung hình; `Texture_Rate` là tốc độ xử lý texture, bằng TMUs nhân xung nhịp.

Các biến về cổng xuất hình, độ phân giải, phiên bản DirectX và OpenGL hay giá bán không liên quan tới câu hỏi nghiên cứu nên bị bỏ. `Integrated` trùng thông tin với `Dedicated` (ngược nghĩa). `L2_Cache` có nhiều card ghi "0KB", nhiều khả năng là không có số liệu. `SLI_Crossfire` chỉ cho biết card *hỗ trợ* ghép nhiều card, không cho biết dòng đó có phải cấu hình ghép hay không, nên cấu hình ghép được nhận diện qua tên ở bước 5. Bốn biến sau tuy có vẻ hữu ích nhưng cũng bị bỏ: `PSU` (bộ nguồn khuyến nghị) và `Power_Connector` (đầu cắm nguồn phụ) được nhà sản xuất chọn sau khi đã biết công suất của card, nên đưa chúng vào mô hình sẽ làm mô hình dự đoán công suất tốt một cách giả tạo; `Pixel_Rate` bằng số ROPs nhân xung nhịp, tức cũng phản ánh quy mô chip và xung nhịp, vốn đã có trong `TMUs` và `Core_Speed`; `Shader` là phiên bản shader (như 5.0) chứ không phải số nhân xử lý. Biến được chọn theo ý nghĩa, không theo tỉ lệ khuyết, và được chọn trước khi làm sạch để các bước sau chỉ xử lý những biến thật sự dùng đến.

Kết quả: 3406 dòng $\times$ 18 biến (chưa tính `gpu_id`).

3.3 Bước 3: Làm sạch chuỗi

Các biến được chọn chứa nhiều giá trị không chuẩn. Ô khuyết của `Core_Speed` ghi `"\n- "` (907 ô) hoặc chỉ chứa ký tự xuống dòng (29 ô); ô khuyết của `Release_Date` ghi `"\nUnknown Release Date "` (30 ô); các ngày còn lại cũng có ký tự xuống dòng và khoảng trắng thừa, như `"\n01-Mar-2009 "`. Ngoài ra có 1 dòng ở biến `Name` bị lỗi mã hóa ký tự (`"Â²"` thay cho `"²"`). Bước này làm ba việc theo thứ tự:

1. xóa khoảng trắng và ký tự xuống dòng ở hai đầu mỗi giá trị, để `"\n01-Mar-2009 "` thành `"01-Mar-2009"` và một ô chỉ có ký tự trắng thành chuỗi rỗng;
2. đổi chuỗi rỗng, `"-"` và `"Unknown Release Date"` thành `NA`;
3. sửa lỗi mã hóa `"Â²"` thành `"²"`.

Đoạn code 2: Xóa khoảng trắng và đánh dấu giá trị khuyết

```
text = selected.apply(lambda column: column.str.strip())
text = text.replace(["", "-"], pd.NA)
text["Release_Date"] = text["Release_Date"].replace(
   "Unknown Release Date", pd.NA)
```

Kết quả: 6474 ô được đánh dấu NA; các ô còn lại là chuỗi không còn khoảng trắng thừa.

Nhận xét. Nếu đọc tệp theo mặc định, pandas đếm được 0 giá trị khuyết ở Core_Speed và Release_Date, vì các ô khuyết ở đây chứa chuỗi chứ không để trống. Vì vậy mọi tỉ lệ khuyết trong mục này đều được tính sau bước 3.

3.4 Bước 4: Xử lý định dạng

Sau bước 3, các biến số vẫn là chuỗi có kèm đơn vị, như "141 Watts" hay "64GB/sec". Với mỗi biến số, nhóm viết một mẫu định dạng riêng, tách phần số ra và đổi sang kiểu số; giá trị nào không khớp mẫu thì chương trình dừng và báo lỗi, nên không có giá trị nào bị đổi sai một cách âm thầm. Riêng ROPs có thể ghi dạng "a (xN)", nghĩa là $a \times N$; ở các dòng này TMUs và Memory_Bus bị ghi chia cho N và được nhân lại (xem phần sửa lỗi bên dưới). Cách xóa mọi ký tự không phải chữ số tuy đơn giản hơn nhưng cho kết quả sai ở hai trường hợp: "800MB/sec" còn lại 800 và bị hiểu là 800 GB/s thay vì 0,8 GB/s; "16 (x2)" còn lại các chữ số "162" thay vì $16 \times 2 = 32$. Bảng 1 liệt kê cách đổi của từng biến.

Bảng 1: Cách chuyển đổi từng biến ở bước 4

Biến gốc	Ví dụ gốc	Cách đổi	Cột mới
Max_Power	141 Watts	bỏ "Watts", đổi sang số	max_power_w
Core_Speed	738 MHz	bỏ "MHz", đổi sang số	core_speed_mhz
Boost_Clock	1100 MHz	như trên	boost_clock_mhz
Memory_Speed	1000 MHz	như trên	memory_speed_mhz
Memory	1024 MB	bỏ "MB", đổi sang số	memory_mb
Memory_Bandwidth	64GB/sec	bỏ đơn vị, đổi sang số	memory_bandwidth_gb_s
	800MB/sec	bỏ đơn vị, chia 1000	
Process	55nm	bỏ "nm", đổi sang số	process_nm
Texture_Rate	47 GTexel/s	bỏ đơn vị, đổi sang số	texture_rate_gtexel_s
ROPs	16 (x2)	lấy hệ số N (không có thì $N = 1$)	(dùng để sửa lỗi)
TMUs	64	đổi sang số, nhân N	tmus
Memory_Bus	256 Bit	bỏ "Bit", đổi sang số, nhân N	memory_bus_bit
Release_Date	01-Mar-2009	đọc dạng ngày, lấy năm	release_year
Dedicated	Yes / No	Yes $\rightarrow$ đúng, No $\rightarrow$ sai	is_dedicated
Notebook_GPU	Yes / No	như trên	is_notebook
Manufacturer	Nvidia	chuyển sang kiểu phân loại	manufacturer
Memory_Type	GDDR5	như trên	memory_type
Name	GeForce GTS 150	giữ nguyên dạng chuỗi	name
Architecture	Tesla G92b	như trên	architecture

Nhận xét.

- 4 dòng ghi băng thông theo MB/sec được đổi sang GB/s (chia 1000, vì nhà sản xuất ghi băng thông theo hệ thập phân).
- Release_Date gốc là chuỗi gồm ngày, tháng viết tắt tiếng Anh và năm, nối bằng dấu gạch, như "01-Mar-2009". Chuỗi được đọc thành kiểu ngày tháng theo đúng mẫu này, rồi lấy phần năm và lưu thành số nguyên release_year (1998–2017 trong dữ liệu cuối). Nhóm chỉ giữ năm vì các phân tích sau chia mẫu theo năm, không cần đến tháng; các ô NA vẫn là NA.
- Dedicated và Notebook_GPU gốc là chuỗi "Yes" hoặc "No", được đổi sang kiểu đúng/sai (is_dedicated, is_notebook) để dùng trực tiếp trong điều kiện lọc và khi đếm.

`Dedicated` có 14 ô khuyết, giữ là `NA` và bị loại ở bước 5. `Notebook_GPU` không có ô khuyết (709 Yes, 2697 No) và được dùng ở bước 6 để tạo `segment`.

- `Manufacturer` và `Memory_Type` là biến định tính, được chuyển sang kiểu phân loại (`category`, tương đương `factor` trong R), tức mỗi giá trị khác nhau là một nhóm. `Manufacturer` có 4 nhóm (Nvidia, AMD, Intel, ATI); Intel chỉ có 7 card trong dữ liệu cuối, nên cách mã hóa biến này trong mô hình hồi quy do phần hồi quy quyết định. `Memory_Type` có 12 nhóm trong dữ liệu gốc (11 nhóm trong dữ liệu cuối) và 56 ô khuyết; nhiều nhóm rất nhỏ nên được gộp lại ở bước 6.

Sửa lỗi "(xN)". 305 dòng ghi `ROPs` dạng "a (xN)". Đây là phép nhân: Radeon RX 580 ghi `"16 (x2)"` và có $16 \times 2 = 32$ ROPs. Ở đúng các dòng này, `TMUs` và `Memory_Bus` bị ghi chia cho N. Ví dụ dòng GTX 1080 Ti Zotac Founders Edition ghi ROPs là `"22 (x4)"` và 56 TMUs, trong khi dòng GeForce GTX 1080 Ti (bản tham chiếu) ghi 88 ROPs và 224 TMUs; nhân lại với $N = 4$ được đúng 224. Có hai bằng chứng, đều do hàm `check_consistency` kiểm tra. Một dòng được coi là không thỏa nếu hai vế lệch nhau quá 10% (với tốc độ texture, cho phép thêm sai số làm tròn 0,5 GTexel/s, vì tệp gốc ghi giá trị này dạng số nguyên):

- Tốc độ xử lý texture bằng TMUs nhân xung nhịp (xung nhịp tăng tốc nếu có, nếu không thì xung nhịp cơ bản). Trong 305 dòng có hệ số N, 274 dòng không thỏa đẳng thức này; 31 dòng còn lại không có xung nhịp nên không kiểm tra được. Sau khi nhân lại với N, còn 0 dòng không thỏa.
- Băng thông bộ nhớ bằng độ rộng bus nhân tốc độ bộ nhớ, nhân một hệ số theo loại bộ nhớ. Hệ số này được ước lượng bằng trung vị của từng loại bộ nhớ, nên chính các dòng lỗi cũng làm lệch nó. Trước khi sửa có 312 dòng không thỏa: toàn bộ 305 dòng có hệ số N và 7 dòng khác. Sau khi sửa, 3 trong 7 dòng khác này thỏa lại vì hệ số hết bị lệch; còn 4 dòng không thỏa, đều là cấu hình SLI cho laptop bị loại ở bước 5.

Theo nguyên tắc làm sạch dữ liệu, giá trị sai được sửa khi tìm được giá trị đúng [1]; ở đây giá trị đúng tính được từ chính các cột khác của dòng đó. Nếu không sửa, các card bị lỗi sẽ bị xem như card nhỏ hơn nhiều so với thực tế trong mô hình hồi quy, vì `TMUs` là một biến giải thích. Phần lớn các dòng có hệ số N là cấu hình nhiều GPU và bị loại ở bước 5; dữ liệu cuối còn 100 dòng đã được sửa, đều là card phát hành năm 2016–2017. `Max_Power` không bị chia cho N: ở hai chip có cả dòng bị lỗi lẫn dòng không lỗi (GP102, ghi trong dữ liệu là "Pascal P102", và Baffin PRO), công suất trung vị của hai loại dòng bằng nhau (220 W và 75 W).

Đoạn code 3: Sửa lỗi "(xN)": nhân lại TMUs và Memory_Bus với N

```
multiplier = parse_multiplier(selected["ROPs"]) # "16 (x2)" -> 2, "64" -> 1
tmus = parse_number(selected["TMUs"]) * multiplier
memory_bus_bit = parse_number(selected["Memory_Bus"], "Bit") * multiplier
```

Kết quả: vẫn 3406 dòng; mọi biến số đã ở kiểu số với một đơn vị thống nhất, và số ô `NA` của mỗi biến không đổi so với bước 3 (riêng `ROPs` được thay bằng hệ số N, ô khuyết cho $N = 1$).

3.5 Bước 5: Lọc quan sát

Đề tài nghiên cứu *một GPU rời đã bán ra thị trường*, nên bước này lần lượt loại:

1. GPU tích hợp (`Dedicated` là No) và các dòng không rõ loại GPU, vì GPU tích hợp nằm chung chip với CPU nên công suất ghi cho chúng không tách khỏi CPU và không so sánh được với GPU rời;

2. các cấu hình nhiều GPU, vì công suất ghi cho các cấu hình này là của cả hệ. Chúng được nhận diện bằng các quy tắc trên tên sản phẩm: tên chứa "SLI" hoặc "CrossFire"; card hai chip có "X2" ngay sau số hiệu (như R9 295X2, HD 4870 X2) hoặc "GX2"; cấu hình ghép GPU tích hợp với GPU rời, tên có dạng "A + B"; và một danh sách các sản phẩm nhiều GPU đã biết (như GTX 690, Quadro Plex). Các tên có "X2" chỉ là tên dòng tản nhiệt (như HIS IceQ X2, Inno3D Twin X2) vẫn được giữ. Danh sách các tên khớp quy tắc đã được rà lại để tránh loại nhầm;

3. card chưa phát hành (tên có ghi "[not released]");

4. các dòng trùng nhau hoàn toàn.

Bảng 2: Số dòng còn lại sau từng điều kiện lọc

Điều kiện lọc	**Số dòng**
Dữ liệu gốc	3406
Chỉ giữ GPU rời	3018
Loại cấu hình nhiều GPU	2775
Loại card chưa phát hành	2762
Loại dòng trùng hoàn toàn	2762

Nhận xét. Theo Bảng 2, điều kiện thứ nhất loại 374 GPU tích hợp và 14 dòng không rõ loại GPU; các điều kiện sau loại 243 cấu hình nhiều GPU và 13 card chưa phát hành. Không có dòng trùng hoàn toàn. Dữ liệu còn 2762 GPU rời, gồm 2318 card desktop và 444 card laptop.

Đơn vị quan sát là một mẫu card bán ra thị trường, không phải một chip. Nhiều hãng gia công bán card dùng chung một chip và thường ghi cùng một công suất định mức (ví dụ phần lớn các bản GTX 1080 Ti ghi 220 W), nên các quan sát không hoàn toàn độc lập. Hạn chế này cần được lưu ý khi diễn giải các kiểm định ở các mục sau.

3.6 Bước 6: Tạo biến mới

Bước này tạo các biến mà nhiều phân tích phải dùng cùng một định nghĩa, rồi giữ lại 16 cột cho bộ dữ liệu cuối (Bảng 3):

- `segment`: Desktop hoặc Laptop, lấy từ `is_notebook`.

- `needs_aux_power`: công suất trên 75 W, mức tối đa một khe PCIe cấp được, tức card cần thêm dây nguồn phụ. Cột này chỉ có ý nghĩa với card desktop, vì card laptop không cắm qua khe PCIe (có 41 card laptop trên 75 W); khi dùng cột này cần chỉ xét card desktop.

- `memory_type_group`: loại bộ nhớ, gộp các loại có ít hơn 30 card biết công suất (DDR, DDR4, GDDR2, GDDR4, HBM-1, HBM-2) thành "Other", còn 6 nhóm. Ngưỡng 30 giúp mỗi nhóm đủ lớn để so sánh giữa các nhóm. Nhóm "Other" gộp các công nghệ rất khác nhau (từ DDR đời cũ tới HBM) nên không được diễn giải riêng.

- `texture_rate_per_watt`: hiệu quả năng lượng, bằng tốc độ xử lý texture chia cho công suất.

Các cột chỉ dùng trong lúc xử lý (như `is_dedicated`, `memory_bus_bit`, `boost_clock_mhz`) không được giữ lại.

Bảng 3: Các cột của bộ dữ liệu sau tiền xử lý: kiểu dữ liệu và tỉ lệ khuyết

Vai trò	**Cột**	**Kiểu dữ liệu**	**Khuyết (%)**
Định danh	`gpu_id`	số nguyên	0,0
	`name`	chuỗi	0,0
Biến mục tiêu	`max_power_w`	số thực	17,4
Biến tạo từ mục tiêu	`needs_aux_power`	đúng/sai	17,4
	`texture_rate_per_watt`	số thực	24,2
Biến giải thích	`tmus`	số đếm	13,0
	`core_speed_mhz`	số thực	22,0
	`memory_bandwidth_gb_s`	số thực	1,3
	`memory_mb`	số thực	0,9
	`process_nm`	số thực	10,5
Biến phân nhóm	`manufacturer`	phân loại	0,0
	`segment`	phân loại	0,0
	`memory_type`	phân loại	0,7
	`memory_type_group`	phân loại	0,7
Thời gian	`release_year`	số nguyên	1,1
Kiến trúc chip	`architecture`	chuỗi	1,5

Kết quả: bộ dữ liệu cuối gồm 2762 GPU rời và 16 cột. Bảng 4 là một vài dòng của bộ dữ liệu: mỗi biến số đã ở dạng số với một đơn vị, giá trị khuyết ghi là `NA`, và dòng GTX 1080 Ti Zotac Founders Edition đã có số TMUs đúng.

Bảng 4: Một vài dòng của bộ dữ liệu sau tiền xử lý

Tên	**Năm**	**Công suất** (W)	**TMUs**	**Xung nhịp** (MHz)	**Băng thông** (GB/s)	**Bộ nhớ** (MB)	**Tiến trình** (nm)
GeForce 8800 GT	2007	105	56	600	57,6	512	65
GeForce 9600 GS	2008	NA	24	NA	24,0	768	65
GeForce GTX 1080 Ti Zotac Founders Edition 11GB	2017	220	224	1480	484,4	11264	16
Radeon RX 580 8GB	2017	150	144	NA	256,0	8192	14

3.7 Xử lý dữ liệu khuyết

Cách xử lý. Nhóm không điền giá trị khuyết. Bộ dữ liệu cuối giữ nguyên các ô `NA` (tỉ lệ khuyết của từng cột ở Bảng 3). Nhóm cũng không loại bỏ các dòng khuyết ngay ở bước tiền xử lý, vì tỉ lệ khuyết quá lớn: 924 trong 2762 dòng (33,5%) có ít nhất một ô khuyết, nên nếu loại bỏ hết thì chỉ còn 1838 dòng, và phân tích chỉ cần công suất sẽ mất 443 card dù có đủ số liệu công suất. Thay vào đó, khi làm một phân tích, nhóm chỉ bỏ những dòng bị khuyết ở các biến mà phân tích đó dùng. Ví dụ:

- tính khoảng tin cậy cho công suất trung bình chỉ cần biến công suất, nên dùng được 2281 trong 2762 card;
- phân tích cần công suất và cả năm biến giải thích dùng được 1866 card.

Vì sao không điền. Có ba cách xử lý dữ liệu khuyết: (1) bỏ các dòng khuyết; (2) điền bằng trung bình hoặc trung vị của các giá trị không khuyết; (3) điền bằng một giá trị cố định. Cách (3) không dùng được, vì một giá trị như 0 W không có ý nghĩa với GPU. Để so sánh cách (1) và cách (2), nhóm tính các đặc trưng mẫu của công suất theo hai cách (Bảng 5).

Bảng 5: Đặc trưng mẫu của công suất khi bỏ dòng khuyết và khi điền trung vị

Đặc trưng	**Bỏ dòng khuyết**	**Điền trung vị**
Cỡ mẫu n	2281	2762
Trung bình mẫu $\bar{x}$ (W)	118,9	117,4
Độ lệch chuẩn mẫu s (W)	77,5	70,5
Độ chính xác ε của khoảng tin cậy 95% (W)	3,2	2,6
Tỉ lệ card có công suất bằng trung vị (%)	1,0	18,2

Nhận xét. Trung bình mẫu gần như không đổi, nhưng độ lệch chuẩn giảm từ 77,5 xuống 70,5 W và độ chính xác ε giảm từ 3,2 xuống 2,6 W. Lý do là 481 giá trị điền vào đều bằng trung vị, nên 18,2% card có cùng một công suất và dữ liệu trông ít phân tán hơn thực tế. Khi đó khoảng tin cậy hẹp hơn mức đúng, và kiểm định dễ bác bỏ H_0 khi H_0 đúng (sai lầm loại I). Ngoài ra, công suất là biến mục tiêu của đề tài; điền giá trị vào là tự tạo ra chính số liệu cần phân tích. Vì vậy nhóm chọn cách (1).

Lưu ý. Cách bỏ dòng khuyết chỉ cho kết quả đúng nếu card bị khuyết giống các card còn lại. Nhóm kiểm tra điều này bằng hai kiểm định hai mẫu, mức ý nghĩa 5%:

- năm phát hành trung bình của card khuyết công suất là 2009,0, của card có công suất là 2012,7; kiểm định so sánh hai trung bình cho $z = -19{,}17$, $p < 0{,}001$;
- trong số card Nvidia và AMD, tỉ lệ card AMD ở nhóm khuyết công suất là 53,1%, ở nhóm có công suất là 39,0%; kiểm định so sánh hai tỉ lệ cho $z = 5{,}65$, $p < 0{,}001$. ATI (83 card, 0 card khuyết công suất) và Intel (7 card) không đưa vào phép so sánh này.

Cả hai kiểm định đều bác bỏ H_0: card khuyết công suất là card đời cũ hơn và có nhiều card AMD hơn. Đây là thiên kiến không phản hồi, tức những phần tử thiếu số liệu có đặc điểm khác những phần tử có số liệu. Vì vậy các kết quả về công suất đại diện nhiều hơn cho card đời mới.

3.8 Các hàm chính và cấu trúc dữ liệu

Dữ liệu được lưu dưới dạng bảng `pandas.DataFrame`: mỗi dòng là một mẫu card, mỗi cột là một biến với kiểu dữ liệu như ở Bảng 3. Bảng 6 giải thích các hàm chính, theo thứ tự trong sơ đồ khối ở Hình 1.

Bảng 6: Các hàm chính trong quy trình tiền xử lý

Hàm	**Chức năng**
`read_raw`	Đọc tệp gốc, mọi cột dưới dạng chuỗi
`select_columns`	Giữ các biến cần cho đề tài
`clean_text`	Xóa khoảng trắng, đánh dấu giá trị khuyết, sửa lỗi mã hóa
`parse_columns`	Tách đơn vị, đổi kiểu, sửa lỗi "(xN)"; báo lỗi nếu gặp định dạng lạ
`check_consistency`	Đếm các dòng không thỏa đẳng thức giữa các biến
`filter_rows`	Lọc quan sát, đếm số dòng còn lại sau mỗi điều kiện
`derive_variables`	Tạo biến mới, giữ các cột cần dùng

References

[1] J. Van den Broeck, S. Argeseanu Cunningham, R. Eeckels, and K. Herbst, "Data cleaning: Detecting, diagnosing, and editing data abnormalities," *PLoS Med.*, vol. 2, no. 10, p. e267, 2005, doi: 10.1371/journal.pmed.0020267.